Talka Your files, our magic.

Technical Test — Document Extraction & Translation

What is this test about?

We need a program that takes a **scanned document** (a PDF that is essentially a photograph — not selectable text) and does two things automatically:

1. **Reconstructs it as an editable Word document (.docx)** — extracting all the text from the image and rebuilding the document as faithfully as possible to the original: same layout, fonts, spacing, and visual elements.
2. **Produces a second Word document with the text translated into Spanish** — replacing the English text with its Spanish equivalent, while keeping the same structure and formatting.

The reason these matters: in a real environment, this process runs on **more than 100 documents per day**, each up to 40 pages long. So, the solution needs to work with minimal human involvement — it cannot rely on someone manually copying, pasting, or fixing things file by file.

The document provided for this test is a **2-page scanned PDF in English**. Think of it as a small-scale version of what the production system would handle.

What you need to build

Step 1 — Extract and reconstruct (English DOCX)

Write a program that reads the scanned PDF and produces an editable .docx file where:

- The **text is complete and accurate** — nothing missing, nothing added, nothing misread.
- The **layout matches the original** as closely as possible: headings, paragraphs, tables, spacing, and font styles.
- **Images, signatures, and stamps** from the original are copied into the output document (not described — actually embedded as images).

Step 2 — Translate (Spanish DOCX)

Using the extracted text from Step 1, produce a second .docx file where:

- All text has been **translated from English to Spanish**.

- The **layout and formatting are identical** to the English DOCX from Step 1.
- Translation can be done manually or with an AI tool (e.g. DeepL, ChatGPT, Claude API). **Using AI for translation is a bonus** — but what we're really evaluating is whether the translated text ends up correctly placed in the right spots in the document, not the translation quality itself.

Technical requirements

- **Language:** Java. This is the primary stack, so the core logic must be in Java. If you use a small helper script in another language for a specific task, that's fine — just explain why.
- **External tools and services:** You are free to use anything — OCR services, document APIs, AI models, open source libraries. If a tool requires a paid plan, free trials are acceptable. If you use something we might not have, just document it so we understand what it does and why you chose it.
- **Automation:** The goal is to minimize manual steps. Every step that requires a human to do something by hand is a step that doesn't scale.

What to submit

- **Source code** — the full project, runnable from scratch.
- **Output files** — the two .docx files produced from the test PDF.
- **A diagram or description of how the process works** — a simple flowchart, a diagram, or even a written explanation of each step. Any format is fine (PDF, PNG, draw.io, a README section). The goal is to make it easy to understand what happens between input and output.
- **Any other documentation** you think helps explain your decisions, the tools you used, or problems you ran into.

Deliver as a .zip file or a link to a Git repository (GitHub, GitLab, etc.).

How your work will be evaluated

What we look at	Why it matters
Text accuracy — the extracted text matches the original exactly, with no missing words, extra words, or misreadings	Errors in extraction propagate to everything downstream

Visual fidelity — the DOCX looks like the original: layout, formatting, images, stamps, signatures	The output is used directly; it needs to be usable
Reusability — how much work would it take to run this on a different but similar document?	The solution needs to work across many files, not just this one
Automation — how many manual steps are required per file?	At 100+ files/day, manual steps are not an option
AI-assisted translation	Bonus point

Deadline

Described in the email. If you have questions, ask them before the deadline — not during the final call.

Send an email to: tech@taikacore.com

Live presentation

After you submit, we will schedule a short video call with the technical team (in Spanish, or English if you prefer). During the call you will:

- **Walk us through your solution** — how it works, step by step.
- **Talk about the challenges** you ran into and how you solved them (or didn't).
- **Explain your tool choices** — why you picked each service or library.
- **Run the pipeline live** on a new document we will provide at the time of the call, similar to the test file.

There are no trick questions. We want to understand how you think and how your solution behaves on a real input.